

Java代码审计基础-JSP核心技术

目录大纲

- JSP基本概念
 - 什么是JSP
 - 为什么要使用JSP
 - 如何使用JSP
- JSP的工作原理
 - JSP与Servlet的区别
 - JSP生命周期
 - JSP的九大内置对象
- JSP的使用
 - JSP语法
 - JSP指令
 - JSP行为
- EL表达式
 - 什么是EL表达式
 - 为什么要用EL表达式
 - EL表达式的使用
 - EL表达式内置的11个对象
 - 扩展EL表达式功能
- JSTL标签库
 - 什么是JSTL标签库
 - 为什么要用JSTL标签库
 - 如何使用JSTL标签库
 - SQL标签库
 - fmt标签库
 - fn方法库
 - 自定义标签

一、JSP基本概念

1.1 什么是JSP

JSP（全称Java Server Pages），根据英文直译过来便是：Java服务器页面

文件主体是HTML，可以插入Java语法以及JSP特定的语法结构

1.2 为什么要用JSP

为了将Servlet中的HTML代码分离，在Java初期，Servlet想要输出HTML必须使用HttpServletResponse对象，Servlet代码与HTML混合在一起，会带来2个问题

1. 代码耦合在一起，不好维护
2. 前后端开发人员需要共同编辑一个Servlet文件，降低了开发效率

在任何行业，只要是影响生产效率的问题都将被优先解决

1.3 如何使用JSP

先看一个简单的实例

```
1  <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2  <%
3      pageContext.setAttribute("name", "Margin");
4  %>
5  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
6      "http://www.w3.org/TR/html4/loose.dtd">
7  <html>
8  <head>
9      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
10     <title>OGNLTTest</title>
11 </head>
12 <body>
13 <h2>Ognl Demo</h2>
14 <form method="post" action="login">
15     <input name="username" type="text" />
16     <input name="password" type="text" />
17     <input type="submit" value="提交" />
18 </form>
19 </body>
20 </html>
```

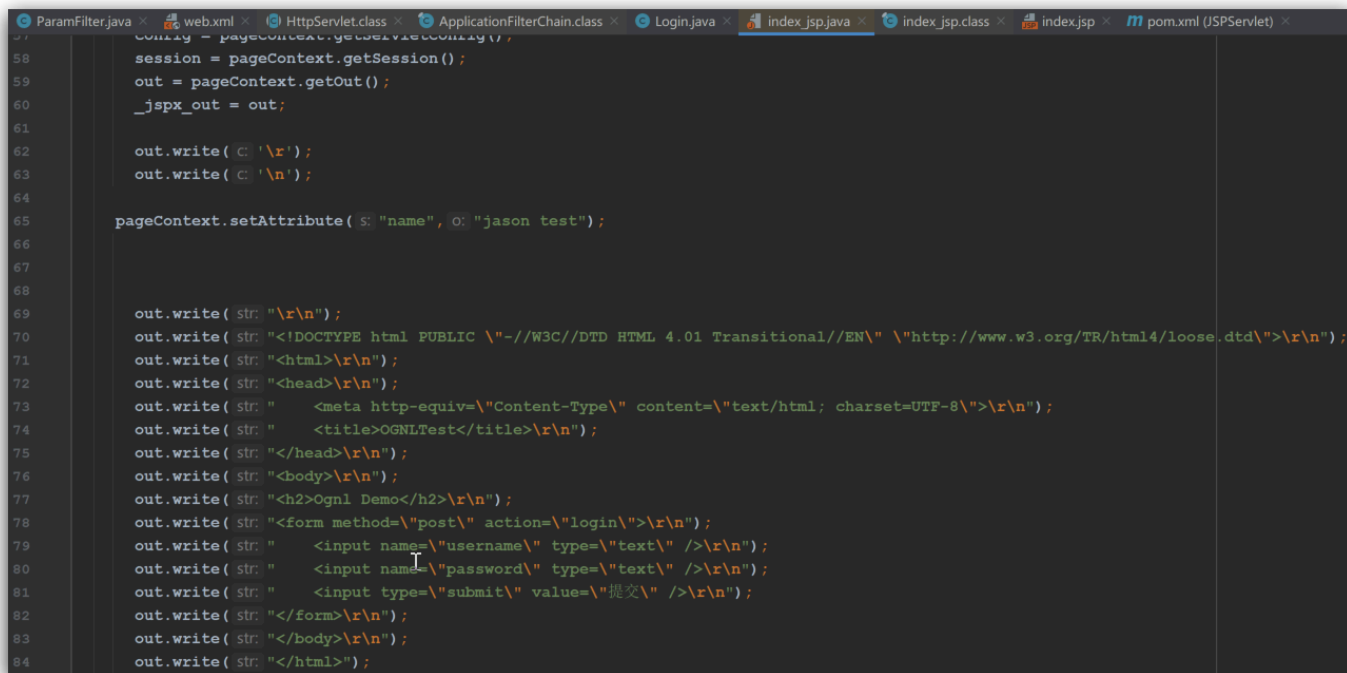
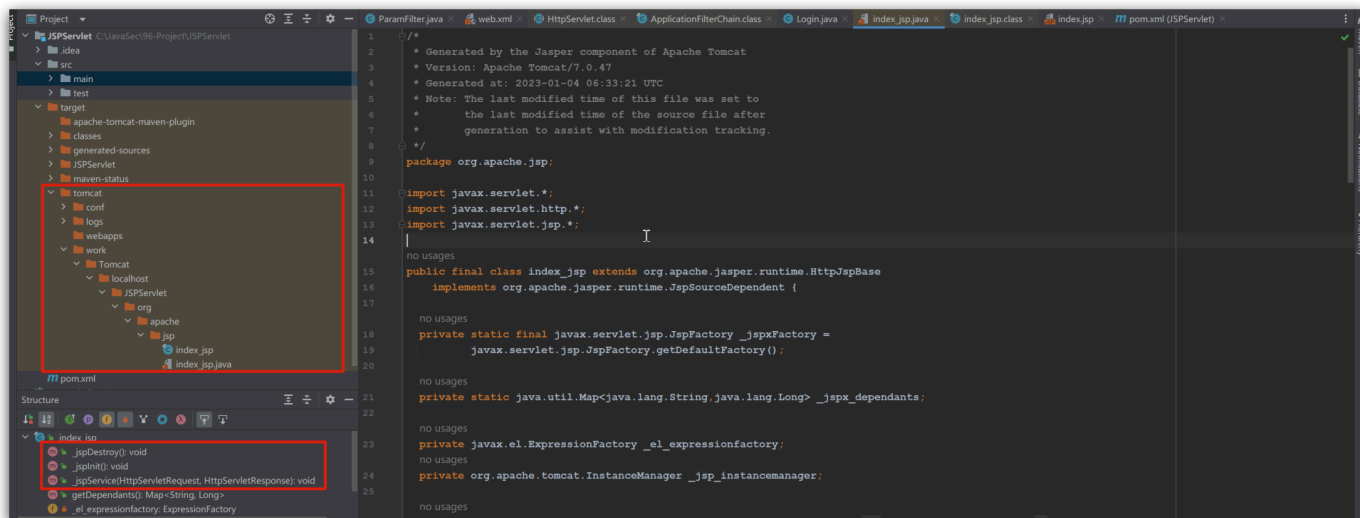
整个的文件结构就是HTML，然后添加了Java代码，要使用JSP需要2个步骤

1. 添加 `javax.servlet.jsp-api` 库
2. 编写JSP页面

二、JSP的工作原理

2.1 JSP与Servlet的区别

当浏览器请求JSP页面时，Web容器将请求交给JSP引擎处理（JspServlet），当JSP第一次被访问时，JSP引擎会将其编辑为class文件，然后Web容器再去调用，路径为 `tomcat/work/Tomcat/localhost/项目名称/包路径`，如下图



由此可见JSP本质上就是Servlet+HTML，里面有3个方法，分别是 `_jspInit`、`_jspDestroy`、`_jspService`，是不是很像Servlet中的 `init`、`destroy`、`service` 方法

2.2 JSP生命周期

因为JSP也是一个Servlet，所以也遵循Servlet的基本规范，JSP初始化和销毁时也会调用Servlet的 `init()` 和 `destroy()` 方法，JSP也有自己的初始化和销毁方法，分别为 `_jspInit`、`_jspDestroy`

2.2 JSP的九大内置对象

什么是内置对象，就是JSP引擎默认创建的对象，可以方便的在JSP页面的Java代码中使用，例如文章开始时的JSP页面案例，页面中并未定义 `pageContext`，但却可以直接使用，这就是内置对象，共九个

内置对象名称	作用	作用域
pageContext	可以获取JSP页面的out、request等八大对象	page
page	代表当前JSP页面本身	page
config	获取服务器的配置信息，等于pageContext.getServletConfig()	application
request	HttpServletRequest对象	request
response	HttpServletResponse对象	request
session	服务器创建的与用户相关的session对象	session
application	application对象内的变量在整个服务器都有效	application
exception	异常对象，当isErrorPage="true"时才可以使用	page
out	用于在浏览器输出信息	page

1. **page**：只在一个页面中保存属性，跳转页面无效
2. **request**：只在一次请求中保存属性，服务器跳转有效，浏览器跳转无效
3. **session**：在一个会话范围中保存属性，无论何种跳转均有效，关闭浏览器后无效
4. **application**：在整个服务器中保存，所有用户都可以使用

三、JSP的使用

3.1 JSP语法

前面介绍了JSP就是：HTML + Servlet，对应到代码上是：

- HTML代码
- JSP语法：`Java代码`、`JSP指令`、`JSP标签`

JSP中的Java代码定义的四种方式：

- `<%%>`：定义局部变量
- `<%!%>`：定义类和方法，几乎很少有人这么用
- `<%= %>`：表达式输出，输出各种类型的变量，int、double、String、Object等
- `<jsp:scriptlet>`：与`<%%>`功能相同

JSP的注释：

- `<%-- %>` JSP注释
- `//` Java单行注释

- `/**/` Java多行注释

3.1 JSP指令

JSP指令用于声明JSP页面的相关属性，例如编码、文档类型等

page指令：

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
```

语法为：

```
1 <%@指令 属性名="值" %>
```

include指令：

在网页中，有些共同的页面例如头、尾，可以使用Include指令包含到某个JSP页面，简化代码

```
1 <%@include file="head.jsp" %>
2 <%@include file="foot.jsp" %>
```

taglib指令：

Taglib 指令是定义一个标签库以及其自定义标签的前缀。一个自定义的tag标签是用户定义的一种JSP标记。当一个含有自定义的tag标签的JSP页面被jsp引擎编译成servlet时，tag标签被转化成了对一个称为tag处理类的对象进行的操作。当JSP页面被jsp引擎转化为servlet后，实际上tag标签被转化成为了对tag处理类的操作

```
1 <%@ taglib uri="" prefix="c" %>
```

1. prefix 是一个标签库别名
2. taglib的uri 引入jsp的标签库

例如使用JSTL标签库，首先引入标签库（当然也需要JSTL的jar包以及taglibs的jar包，后面再讲）

```
1 <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

引入后使用

```
1 <c:out value="${name}"
```

关于JSP标签库的内容，在下面有专门的章节讲解

3.1 JSP行为

JSP行为其实是一组JSP内置的标签，对常用的JSP功能进行了封装，它的 **prefix** 是 **jsp**，例如

include行为

```
1 <jsp:include page="head.jsp"/>
2 <jsp:include page="foot.jsp"/>
```

forward行为

在Servlet中HttpServletRequest可以通过request.getRequestDispatcher(String url).forward(request,response)进行跳转，**forward行为就是对其封装**

```
1 <jsp:forward page="" />
```

param行为

当使用<jsp:include>和<jsp:forward>行为引入或将请求转发给其它资源时，可以使用<jsp:param>行为向这个资源传递参数

```
1 <jsp:forward page="head.jsp">
2     <jsp:param name="id" value="1" />
3 </jsp:forward>
```

directive行为

directive的中文意思就是指令，该行为就是替代指令<%@%>的语法

- <jsp:directive.include file="" /> 相当于<%@include file="" %>
- <jsp:directive.page /> 相当于<%@page %>
- <jsp:directive.taglib /> 相当于<%@taglib %>

```
1 <jsp:directive.include file="head.jsp"></jsp:directive.include>
2 <jsp:directive.include file="foot.jsp"></jsp:directive.include>
```

四、EL表达式

4.1 什么是EL表达式

EL (Expression Language)，直译就是**表达式语言**，表达式使用 **`${}`** 包裹，OGNL表达式是使用 **`%{}`**，不要记混了

- EL表达式只能读取数据，不能修改数据
- EL表达式只能处理四大域中的属性值及常量

4.2 为什么要用EL表达式

假设如果没有EL表达式，想访问application对象的值，如何访问？

```
1  <%@ page contentType="text/html;charset=UTF-8" language="java" %>
2  <html>
3  <head>
4      <title></title>
5  </head>
6  <body>
7
8  <%
9      String value = (String) application.getAttribute("name");
10     out.write(value);
11 %>
12 </body>
13 </html>
```

可以看出很麻烦，那如果使用EL表达式呢？

```
1  <%@ page contentType="text/html;charset=UTF-8" language="java" %>
2  <html>
3  <head>
4      <title></title>
5  </head>
6  <body>
7
8      ${applicationScop.name}
9
10 </body>
11 </html>
```

通过对比就很清晰的理解EL表达式的作用，就是简化代码、提升开发效率

4.3 EL表达式的使用

EL表达式的查找属性的顺序按照作用域由小到大的顺序进行查找，也就是 `application` > `session` > `request` > `pageContext`，在Servlet基础中讲过，Servlet中只有三个域，没有pageContext域

获取JSP域对象的属性

JSP的域对象	EL表达式
Application	<code>\${applicationScop.属性名}</code>
Session	<code>\${sessionScop.属性名}</code>
Request	<code>\${requestScop.属性名}</code>
pageContext	<code>\${pageScop.属性名}</code>

```

1 <%
2     pageContext.setAttribute("username","北京");
3     request.setAttribute("username","上海");
4     session.setAttribute("username","重庆");
5     application.setAttribute("username","武汉");
6 %>
7 username = ${pageScope.username}<br>
8 username = ${sessionScope.username}<br>

```

获取自定义对象的属性

一般来说都是使用 `对象.属性名` 获取，例如

```

1 <%
2     Student student = new Student("张三",23);
3     pageContext.setAttribute("student",student);
4 %>
5
6 <!--可以获取到student的信息(重写了toString()方法)-->
7 student = ${student}
8 <!--EL表达式属性值要和student类中的属性相对应-->
9 name = ${student.name} <!--获取到student对象的name值-->
10 age = ${student.age} <!--获取到student对象的age值-->
11
12 name = ${student['name']} <!--获取到student对象的name值-->
13 age = ${student['age']} <!--获取到student对象的age值-->

```

EL访问数组

访问普通数组

```

1 <%
2     String[] name = new String[]{"李四","张三","王五"};
3     pageContext.setAttribute("names",name);
4 %>
5 ${names[1]} <!--取出的数据为：张三-->
6 <!--若访问的数组元素的下标超过了数组下标最大值，EL不会输出下标越界异常-->
7 ${names[5]}

```

访问对象数组

```

1 <%
2     Student[] students = new Student[2];
3     students[0] = new Student("张三",20);
4     students[1] = new Student("李四",30);
5     pageContext.setAttribute("students",students);
6 %>
7 ${students[0].name} <!--输出的结果为：张三-->
8 ${students[1].age} <!--输出的结果为：30-->

```

EL访问集合(List)

EL表达式可以通过索引访问List集合，但无法访问Set。因为Set中不存在索引。

```

1 <%
2     List<String> names_arr = new ArrayList();
3     names_arr.add("张三");
4     names_arr.add("李四");
5     names_arr.add("王五");
6     pageContext.setAttribute("names_arr",names_arr);
7 %>
8 <!--第一个输出结果为王五，和上边一样，不会报下标越界异常-->
9 ${name[2]}
10 ${name[200]}

```

EL访问Map

```

1 <%
2     // map集合中存储的是Object类型。可以存储任意类型
3     Map<String,Object> map = new HashMap();
4     map.put("student",new Student("张三",age));
5     map.put("mobile","123456");
6     map.put("age",20);
7     pageContext.setAttribute("map",map);
8 %>
9 <!--这里发生了一个类型的向下转型，将Object类型转换为student类型-->
10 ${map.student.name}
11 ${map.mobile}
12 ${map.age}

```

4.4 EL表达式内置的11个对象

内置对象	类型	作用
pagecontext	pagecontextImpl	获取jsp中的九大内置对象
pagescope	map	获取pagecontext域中的数据
requestscope	map	获取request域中的数据
sessionscope	map	获取session域中的数据
applicationscope	map	获取application域中的数据
param	map	获取请求参数的值
paramvalues	map	获取请求参数的值，获取多个值的时候使用、
header	map	获取请求头的信息
headervalues	map	获取请求头的信息,可以获取多个值的情况
cookie	map	获取当前请求的cookie信息
initParam	map	获取在web.xml中配置的上下文参数

EL表达式中的pageContext对象与JSP中的pageContext对象是同一个，都可以获取request、response、session、servletConfig等对象，但request、response、session、servletConfig等对象并不是EL的内置对象，只能通过pageContext获取

EL表达式获取对象的方法其实是对Java方法的封装，例如`${pageContext.request}`获取request对象。其底层实际调用的pageContext.getRequest()方法

4.5 扩展EL表达式功能

业务的不断发展，早期的公司一般都会自己定义一套EL函数库，去扩展EL功能，提高开发效率，所以自定义EL函数库，就是EL的扩展。

自定义EL函数库，需要以下几个动作

- 1. 创建Java的处理函数，用于EL表达式的处理
- 2. 编写tld文件（tag library descriptor 标签库描述符），注册EL函数，使其可在JSP中被识
- 3. 在JSP中使用

创建Java的处理函数，用于EL表达式的处理

```
1 package org.function;
2
3 public class ElFuction {
4     /**
5      * 反转字符串
6      * @param str
7      * @return
8      */
```

```

9      public static String reverseString(String str) {
10          return (new StringBuilder(str).reverse().toString());
11      }
12
13      /**
14       * 字符串长度
15       * @param str
16       * @return
17       */
18      public static Integer stringLength(String str) {
19          return str.length();
20      }
21  }

```

编写tld文件

```

1  <?xml version="1.0" encoding="UTF-8" ?>
2
3  <taglib xmlns="http://java.sun.com/xml/ns/javaee"
4          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5          xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-jsptaglibrary_2_1.xsd"
6          version="2.1">
7      <!-- 标签库的说明信息 -->
8      <description>This is my taglib</description>
9      <!-- 版本 -->
10     <tlib-version>1.0</tlib-version>
11     <short-name>mars</short-name>
12
13     <!-- 标识这个在互联网上的唯一地址，一般是作者的网站，这个网址可以根部不存在对应的资源，但一定
要是唯一的。这里的值用在将在 taglib 指令中 uri的值-->
14     <uri>http://www.mars.com/mars</uri>
15
16     <!-- 每一个function结点 定义一个EL自定义函数 -->
17     <function>
18         <!--函数的功能描述-->
19         <description>返回一个字符串的长度</description>
20         <!--EL函数名,也就是在JSP页面中使用时的名称， he可以和java代码中的函数名不一样-->
21         <name>stringLength</name>
22         <!--函数所在的类 -->
23         <function-class>org.function.ElFuction</function-class>
24         <!-- 函数的签名 。函数的返回类型和参数类型强烈建议 给出全名，如： java.lang.String --
>
25         <function-signature>java.lang.Integer stringLength(java.lang.String)
</function-signature>
26     </function>
27
28     <function>
29         <description>反转一个字符串</description>

```

```

30         <name>reverseString</name>
31         <function-class>org.function.ElFuction</function-class>
32         <function-signature>java.lang.String reverseString(java.lang.String)
</function-signature>
33     </function>
34
35 </taglib>

```

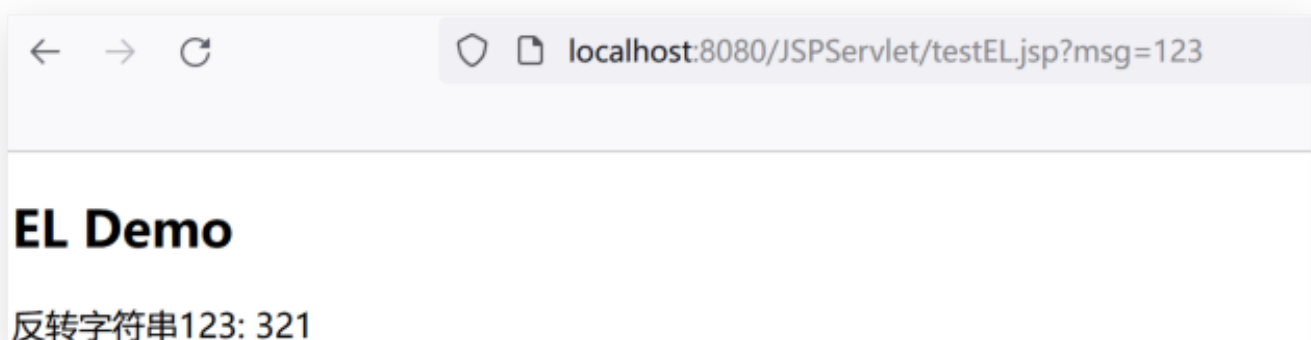
编写JSP文件，一定要加 `<%@ page isELIgnored="false"%>`，表示当前JSP页面是否忽略EL表达式，默认值为false，表示不忽略（即支持）。
如果设定为真，那么JSP中的表达式被当成字符串处理

```

1  <%@ page language="java" contentType="text/html; charset=UTF-8"
2      pageEncoding="UTF-8"%>
3  <%@ page isELIgnored="false"%>
4  <%@taglib uri="http://www.mars.com/mars" prefix="mars"%>
5
6  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
7  <html>
8  <head>
9      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
10     <title>ELTest</title>
11 </head>
12 <body>
13 <h2>EL Demo</h2>
14 反转字符串${param.msg}:  ${mars:reverseString(param.msg)} <br />
15 </body>
16 </html>

```

测试效果如下



五、JSTL标签库

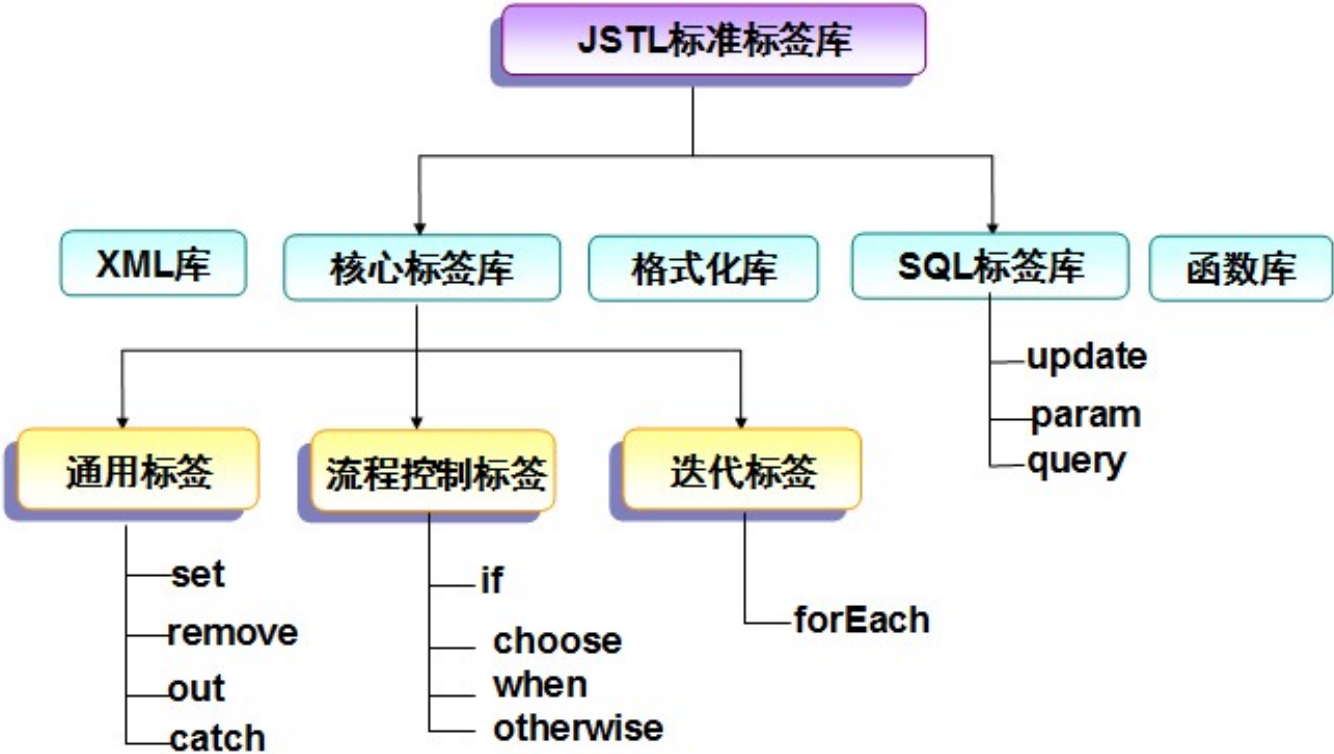
5.1 什么是JSTL标签库？

JSTL (JavaServerPages Standard Tag Library) JSP标准标签库

JSTL由四个定制标记库(core、format、xml 和 sql)和一对通用标记库验证器组成。如果要使用JSTL，则必须引用jstl.jar和 standard.jar两个包，在Maven中需要引入

```
1  <!-- JSTL标签库 -->
2  <dependency>
3      <groupId>jstl</groupId>
4      <artifactId>jstl</artifactId>
5      <version>1.2</version>
6  </dependency>
7  <dependency>
8      <groupId>>taglibs</groupId>
9      <artifactId>standard</artifactId>
10     <version>1.1.2</version>
11 </dependency>
```

JSTL标准标签库内的标签



JSTL是标签，EL是表达式，两者结合JSP页面基本不需要<% %>代码了

5.2 为什么使用JSTL标签库?

正常情况下开发JSP需要再页面中加入非常多的Java代码，这并不优雅，就像最开始的Servlet中写HTML一样。所以JSP中的Java代码增多会带来很多问题

- JSP难维护
- 出错后调试困难
- 前后端分工不明确
- 开发效率低

所以JSP迫切的需要一种可以减少JSP中Java代码的解决方法，从而使开发人员分工明确，提高开发效率，JSTL应运而生

5.3 如何使用JSTL标签库

要使用JSTL需要以下几个步骤

1. 导入Jar依赖（前面已经说过）
2. 在JSP中引入标签库

```
1 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
2 <%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt"%>
3 <%@ taglib uri="http://java.sun.com/jsp/jstl/xml" prefix="x"%>
4 <%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql"%>
5 <%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn"%>
```

3. 如果配合EL使用，JSP中一定要加 `<%@ page isELIgnored="false"%>`

JSTL的核心标签

标签	功能
<c:out>	用于在JSP中显示数据，就像<%= ... >
<c:set>	用于保存数据
<c:remove>	用于删除数据
<c:catch>	用来处理产生错误的异常状况，并且将错误信息储存起来
<c:if>	与我们在一般程序中用的if一样
<c:choose>	本身只当做<c:when>和<c:otherwise>的父标签
<c:when>	<c:choose>的子标签，用来判断条件是否成立
<c:otherwise>	<c:choose>的子标签，接在<c:when>标签后，当<c:when>标签判断为false时被执行
<c:import>	检索一个绝对或相对 URL，然后将其内容暴露给页面
<c:forEach>	基础迭代标签，接受多种集合类型
<c:forTokens>	根据指定的分隔符来分隔内容并迭代输出
<c:param>	用来给包含或重定向的页面传递参数
<c:redirect>	重定向至一个新的URL
<c:url>	使用可选的查询参数来创建一个UR

直接看代码更加清晰

```

1 <c:out value="值或者EL表达式" escapeXML="true|false" default="默认值" />
2 <c:out value="值或者EL表达式" escapeXML="true|false" >默认值</c:out>

```

```

1 <!-- 设置域变量的值 -->
2 <c:set value="值或者EL表达式" var="域变量名" scope="page|request|session|application"/>
3 <c:set var="域变量名" scope="page|request|session|application">值或者EL表达式</c:set>
4 <!-- 设置JavaBean的值 -->
5 <c:set target="EL表达式表示的JavaBean对象" property="属性名" value="属性值" />
6 <c:set target="EL表达式表示的JavaBean对象" property="属性名" >属性值</c:set>

```

```

1 <!-- 资源内容作为String对象被导出 -->
2 <!-- context、var、scope、charEncoding可以缺省 -->
3 <c:import url="url" context="上下文名称" var="资源句柄域变量"
4   scope="page|request|session|application" charEncoding="字符集">
5   可选内容为<c:param>子标签
6 </c:import>
7 <!-- 资源的内容作为Reader对象被导出 -->
8 <!-- context、varReader、charEncoding可以缺省 -->
9 <c:import url="url" context="上下文名称" varReader="资源句柄域变量" charEncoding="字符集">
10   可选内容为调用Reader对象的其他动作
11 </c:import>

```

```

1 <c:import url="a.jsp"/><!-- 使用相对url导入同一上下文的资源 -->
2 <c:import url="/b.jsp"/><!-- 使用相对url导入同一Web容器的资源 -->
3 <c:import url="/hello.jsp" context="/ch17"/><!-- 使用url属性和context属性导入资源 -->
4 <c:import url="/max.jsp" context="/ch17"/><!-- 在内容体中使用<c:param>子标签 -->
5     <c:param name="num1" value="23"/>
6     <c:param name="num2" value="16"/>
7 </c:import>
8 <c:import url="http://www.sunxin.org?keyword=computer"/><!-- 使用绝对路径导入资源 -->
9 <c:import url="ftp://ftp.sunxin.org/copyright"/>

```

```

1 <c:redirect url="value" context="context"/>
2 <c:redirect url="value" context="context">
3     <c:param>子标签
4 </c:redirect>

```

```

1 <c:param name="name" value="value"/>
2 <c:param name="name">
3     value
4 </c:param>

```

格式化标签

标签	描述
<fmt:formatNumber>	使用指定的格式或精度格式化数字
<fmt:parseNumber>	解析一个代表着数字，货币或百分比的字符串
<fmt:formatDate>	使用指定的风格或模式格式化日期和时间
<fmt:parseDate>	解析一个代表着日期或时间的字符串
<fmt:bundle>	绑定资源
<fmt:setLocale>	指定地区
<fmt:setBundle>	绑定资源
<fmt:timeZone>	指定时区
<fmt:setTimeZone>	指定时区
<fmt:message>	显示资源配置文件信息
<fmt:requestEncoding>	设置request的字符编码

5.4 SQL标签库

观察下面的代码，是SQL标签的使用，你觉得简洁吗？我觉得并不简洁。感觉和直接写Java代码并没有什么太大区别

```

1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <!-- 1、导入标签库 -->

```

```

3 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
4 <%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql" %>
5 <%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
6 <html>
7 <head>
8     <title>TestMyJSP</title>
9 </head>
10 <body>
11     <!-- 2、设置数据源 -->
12     <sql:setDataSource url="jdbc:mysql://localhost:3306/helloworld?
useUnicode=true&characterEncoding=utf8&serverTimezone=GMT%2B8&useSSL=false"
13         driver="com.mysql.cj.jdbc.Driver"
14         user="root"
15         password="ljh_5098_CHD" />
16
17     <!-- 3、操作数据库 -->
18     <!-- 利用sql:query标签执行查询 -->
19     <sql:query var="result">
20         select * from student where Sname = ?
21         <sql:param value="藤原拓海" /><!-- 替代sql语句中的? -->
22     </sql:query>
23     <!-- 输出查询结果 -->
24     <table>
25         <c:forEach var="item" items="${result.rows}">
26             <tr>
27                 <td><c:out value="${item.Sno}" /></td>
28                 <td><c:out value="${item.Sname}" /></td>
29                 <td><c:out value="${item.Ssex}" /></td>
30                 <td><c:out value="${item.Sage}" /></td>
31                 <td><c:out value="${item.Sdept}" /></td>
32             </tr>
33         </c:forEach>
34     </table>
35
36     <!-- 利用sql:update标签执行数据更新语句 -->
37     <sql:update>
38         update student set Sname='高桥凉介' where sname='藤原拓海'
39     </sql:update>
40     <!-- 构造一个java.util.Date对象并存储到一个域变量中 -->
41     <fmt:parseDate value="2019-05-29" var="public_date" pattern="yyyy-MM-dd" />
42     <!-- 执行查询语句 -->
43     <sql:query var="result">
44         select *,? date from student<!-- 在查询语句中使用占位符? -->
45         <sql:dateParam value="${public_date}" type="date" /><!-- 使用sql:dateParam标
签将Date对象格式化为一个时间字符串 -->
46     </sql:query>
47     <!-- 输出查询结果 -->
48     <table>
49         <c:forEach var="item" items="${result.rows}">

```

```

50         <tr>
51             <td><c:out value="\${item.date}"/></td>
52             <td><c:out value="\${item.Sno}"/></td>
53             <td><c:out value="\${item.Sname}"/></td>
54             <td><c:out value="\${item.Ssex}"/></td>
55             <td><c:out value="\${item.Sage}"/></td>
56             <td><c:out value="\${item.Sdept}"/></td>
57         </tr>
58     </c:forEach>
59 </table>
60 </body>
61 </html>

```

5.4 fmt标签库

formatting标签库用于在JSP页面中做国际化格式化的动作，也可以通过很简单的方式转换数字、日期，导入格式化标签库的语法如下

标签	描述
<fmt:formatNumber>	使用指定的格式或精度格式化数字
<fmt:parseNumber>	解析一个代表着数字，货币或百分比的字符串
<fmt:formatDate>	使用指定的风格或模式格式化日期和时间
<fmt:parseDate>	解析一个代表着日期或时间的字符串
<fmt:bundle>	绑定资源
<fmt:setLocale>	指定地区
<fmt:setBundle>	绑定资源
<fmt:timeZone>	指定时区
<fmt:setTimeZone>	指定时区
<fmt:message>	显示资源配置文件信息
<fmt:requestEncoding>	设置request的字符编码

5.5 fn标签库

其实EL函数库就是fn方法库，是JSTL标签库中的一个库，也有人称之为fn标签库，但是该库长得不像是标签，所以称之为fn方法库

JSTL 函数（fn）标签大部分是通用的字符串处理函数，如下图

标签	描述
<fn:contains(>	用于判断一个字符串是否包含指定的字符串，区分大小写
<fn:containsIgnoreCase(>	用于判断一个字符串是否包含指定的字符串，不区分大小写
<fn:endsWith(>	用于判断一个字符串是否以指定的后缀结尾
<fn:escapeXml(>	用于转义 HTML/XML 中的字符
<fn:indexOf(>	用于返回字符串在指定字符串中的开始位置
<fn:join(>	用来将数组中的所有元素利用指定的分隔符来连接成一个字符串
<fn:length(>	用于返回指定字符串的长度
<fn:split(>	用于将字符串用指定的分隔符分隔为一个子串数组
<fn:startsWith(>	用于判断一个字符串是否以指定的前缀开头
<fn:substring(>	用来返回指定字符串的子字符串
<fn:substringAfter(>	用来返回字符串中指定子字符串后面的部分
<fn:substringBefore(>	用来返回字符串中指定子字符串前面的部分
<fn:toLowerCase(>	用来将指定字符串中的所有字符转为小写
<fn:toUpperCase(>	将指定字符串中的所有字符转为大写
<fn:trim(>	用来删除指定字符串两端的空格

使用案例

```

1  <%@ page language="java" contentType="text/html; charset=UTF-8"
2      pageEncoding="UTF-8"%>
3  <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
4  <%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn"%>
5  <!DOCTYPE html>
6  <html>
7  <head>
8  <title>FN标签</title>
9  </head>
10 <body>
11     <c:set var="mymsg" value="Example of JSTL function" />
12     字符串以Example开头: ${fn:startsWith(mymsg, 'Example')} <br>
13     字符串以example开头: ${fn:startsWith(mymsg, 'example')} <br>
14     字符串以function结尾: ${fn:endsWith(mymsg, 'function')} <br>
15     字符串以Function结尾: ${fn:endsWith(mymsg, 'Function')} <br>
16 </body>
17 </html>

```

5.6 自定义标签

前面讲EL表达式时可以扩展EL表达式，同样，JSTL也可以扩展自定义的标签，需要以下步骤

1. 创建标签处理的Java类，继承SimpleTagSupport
2. 创建tld标签库描述文件
3. 在JSP中使用

标签的解析会在分析Struts2漏洞时经常用到

创建标签处理的Java类，继承SimpleTagSupport

```
1 package org.function;
2
3 import javax.servlet.jsp.JspException;
4 import javax.servlet.jsp.JspWriter;
5 import javax.servlet.jsp.tagext.SimpleTagSupport;
6 import java.io.IOException;
7
8 public class MarsTag extends SimpleTagSupport {
9     public void doTag() throws JspException, IOException {
10         JspWriter out = getJspContext().getOut();
11         out.println("Hello Mars!!!");
12     }
13 }
```

创建tld标签库描述文件，MarsJSTL.tld

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE taglib
3     PUBLIC "-//Sun Microsystems, Inc.//DTD JSP Tag Library 1.2//EN"
4     "http://java.sun.com/j2ee/dtd/web-jsptaglibrary_1_2.dtd">
5 <taglib>
6     <!-- 标签库版本号 -->
7     <tlib-version>1.0</tlib-version>
8     <!-- JSP版本号 -->
9     <jsp-version>2.0</jsp-version>
10    <!-- 当前标签库的前缀 -->
11    <short-name>mars</short-name>
12    <uri>http://www.mars.com/marsjstl</uri>
13    <tag>
14        <!-- 自定义标签的名称，在页面中通过它来使用标签 -->
15        <name>Hello</name>
16        <!-- 自定义标签的实现类路径 -->
17        <tag-class>org.function.MarsTag</tag-class>
18        <!-- 正文内容正文内容，没有则用 empty 表示 -->
19        <body-content>empty</body-content>
20        <!-- 自定义标签的功能描述 -->
21        <description>输出内容</description>
```

```
22     </tag>
23 </taglib>
```

在JSP中使用

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2     pageEncoding="UTF-8" %>
3 <%@ taglib prefix="mars" uri="http://www.mars.com/marsjstl" %>
4 <!DOCTYPE html>
5 <html>
6 <head>
7     <title>Tag.jsp</title>
8 </head>
9 <body>
10     <mars:Hello/>
11 </body>
12 </html>
13 </html>
```

测试结果

